

ECE264-003 Fall 2019 Exam 2
8-9 PM, Wednesday, Oct 16th, 2019

1. GDB for A Recursive Program

Consider the following program:

```
1.  #include <stdio.h>
2.  #include <stdlib.h>
3.  int func (int n)
4.  {
5.      int rtv;
6.      if (n <= 2)
7.      {
8.          rtv = n;
9.      }
10.     else
11.     {
12.         rtv = n * func (n-2);
13.     }
14.     return rtv;
15. }
16.
17. int main (void)
18. {
19.     int num = 7;
20.     int val = func (num);
21.     printf("Output is: %d\n", val);
22.     return EXIT_SUCCESS;
23. }
```

Suppose you run the following GDB commands; (gdb) is the prompt.
Fill in blanks for the GDB output.

```
(gdb) b main
Breakpoint 1 at 0x100000f4f: file q1.c, line 19.
(gdb) b func
Breakpoint 2 at 0x100000efb: file q1.c, line 6.
(gdb) info b
Num Type          Disp Enb Address                What
1 breakpoint keep y 0x0000000100000f4f in main at q1.c:19
2 breakpoint keep y 0x0000000100000efb in func at q1.c:6
(gdb) r
[New Thread 0x2603 of process 38998]

Thread 2 hit Breakpoint 1, main () at q1.c:19
19     int num = 7;
```

```

(gdb) n
20      int val = func (num);
(gdb) s

Thread 2 hit Breakpoint 2, func (n=1.A) at q1.c:6
6      if (n <= 2)
(gdb) c
Continuing.

Thread 2 hit Breakpoint 2, func (n=5) at q1.c:6
6      if (n <= 2)
(gdb) c
Continuing.

Thread 2 hit Breakpoint 2, func (n=3) at q1.c:6
6      if (n <= 2)
(gdb) c
Continuing.

Thread 2 hit Breakpoint 2, func (n=1) at q1.c:6
6      if (n <= 2)
(gdb) info b
Num  Type          Disp Enb Address          What
1    breakpoint    keep y   0x00000000100000f4f in main at q1.c:19
      breakpoint already hit 1 time
2    breakpoint    keep y   0x00000000100000efb in func at q1.c:6
      breakpoint already hit 1.B times
(gdb) bt
#0  func (n=1) at q1.c:6
#1  0x00000000100000f23 in func (n=3) at q1.c:1.C
#2  0x00000000100000f23 in func (n=5) at q1.c:12
#3  0x00000000100000f23 in func (n=7) at q1.c:12
#4  0x00000000100000f5e in main () at q1.c:20
(gdb) f 2
#2  0x00000000100000f23 in func (n=1.D) at q1.c:12
12      rtv = n * func (n-2);
(gdb) c
Continuing.

Output is: 1.E
[Inferior 1 (process 38998) exited normally]

```

Answer:

1.A: 7

1.B: 4

1.C: 12

1.D: 5

1.E: 105

2. Recursion: Integer Partition

A positive integer can be expressed as the sum of a sequence of positive integers, or itself. And the order of positive integers in the sequence matters. *Integer partition* creates such sequences of integers. Here we consider the case where only **odd** positive integers can be used in the partition. For examples, number 5 has 5 possible partitions.

$$5 = 1+1+1+1+1$$

$$5 = 1+1+3$$

$$5 = 1+3+1$$

$$5 = 3+1+1$$

$$5 = 5$$

Note that:

- 1) $5 = 1+1+3$, $5 = 1+3+1$, and $5 = 3+1+1$ are different partitions since the order of positive integers in the sequence matters.
- 2) Also, $5 = 1+2+2$ is not a valid partition since only odd positive integers can be used.

Question 2.A: How many possible partitions for number 6?

Question 2.B: How many possible partitions for number 7?

Consider the following program to print all possible partitions with odd positive integers for a positive integer n . The executable program's name is `q2prog`.

Question 2.C: Fix the code in line 43.

Question 2.D - 2.E: Write the first two lines of output in 2.D - 2.E when the program is executed in a terminal using `./q2prog 4`

```

1.  #include <stdio.h>
2.  #include <stdlib.h>
3.  void printPartition(int * arr, int length)
4.  {
5.      int ind;
6.      for (ind = 0; ind < length - 1; ind ++)
7.      {
8.          printf("%d + ", arr[ind]);
9.      }
10.     printf("%d\n", arr[ind]);
11. }
12.
13. void partition(int * arr, int ind, int left)
14. {
15.     int val;
16.     if (left == 0)
17.     {
18.         printPartition(arr, ind);
19.         return;
20.     }
21.     for (val = 1; val <= left; val += 2)
22.     {
23.         arr[ind] = val;
24.         partition(arr, ind + 1, left - val);
25.     }
26. }
27.
28. int main(int argc, char * argv[])
29. {
30.     if (argc != 2)
31.     {
32.         return EXIT_FAILURE;
33.     }
34.     int n = (int) strtol(argv[1], NULL, 10);
35.     if (n <= 0)
36.     {
37.         return EXIT_FAILURE;
38.     }
39.     int * arr;
40.     arr = malloc(sizeof(int) * n);
41.     // 2.C: Fix the code to print all possible partitions
42.     // with odd positive integers for a positive integer n
43.     partition(arr, 2.C, n);
44.     free(arr);
45.     return EXIT_SUCCESS;
46. }

```

Answer:

2.A: 8

2.B: 13

2.C: 0

2.D: $1 + 1 + 1 + 1$

2.E: $1 + 3$

3. Recursive Program

Consider the following program.

```
1.  #include <stdio.h>
2.  #include <stdlib.h>
3.  void func (int n)
4.  {
5.      if (n > 0)
6.      {
7.          func (n-1);
8.          printf ("%d ", n);
9.          func (n-1);
10.     }
11. }
12.
13. int main(int argc, char * argv[])
14. {
15.     if (argc != 2)
16.     {
17.         return EXIT_FAILURE;
18.     }
19.     int n = (int) strtol(argv[1], NULL, 10);
20.     func(n);
21.     printf("\n");
22.     return EXIT_SUCCESS;
23. }
```

The executable program's name is q3prog.

3.A: When the program is executed in a terminal using `./q3prog 1`, what is the output?

3.B: When the program is executed in a terminal using `./q3prog 2`, what is the output?

3.C-3.D: When the program is executed in a terminal using `./q3prog 8`, what are the first two numbers in the output?

3.E: When the program is executed using `./q3prog 6`, how many numbers are there in the output?

Answer:

3.A: 1

3.B: 1 2 1

3.C: 1

3.D: 2

3.E: 63

4. Linked List

The following is the definition of a linked list node.

```
1.  // list.h
2.  #ifndef LIST_H
3.  #define LIST_H
4.  typedef struct listnode
5.  {
6.      struct listnode * next;
7.      int value;
8.  } Node;
9.  #endif
```

Consider the following program. Hint: note the return value of each function.

```
1.  // q4.c
2.  #include "list.h"
3.  #include <stdio.h>
4.  #include <stdlib.h>
5.  static Node * Node_construct(int v)
6.  {
7.      Node * n = malloc(sizeof(Node));
8.      n -> value = v;
9.      n -> next = NULL;
10.     return n;
11. }
12.
13. Node * List_insert(Node * head, int v)
14. {
15.     Node * p = Node_construct(v);
16.     p -> next = head; /* insert at beginning */
17.     return p; /* note the return value */
18. }
19.
20. Node * List_search(Node * head, int v)
21. {
22.     Node * p = head;
23.     while (p != NULL)
24.     {
25.         if ((p -> value) == v) { return p; }
26.         p = p -> next;
27.     }
28.     return p; /* note the return value */
29. }
30.
```

```

31. Node * List_delete(Node * head, int v)
32. {
33.     Node * p = head;
34.     if (p == NULL) { return p; } /* do nothing if empty */
35.     if ((p -> value) == v) /* delete the first node*/
36.     {
37.         p = p -> next;
38.         free (head);
39.         return p;
40.     }
41.
42.     Node * q = p -> next;
43.     while ((q != NULL) && ((q -> value) != v))
44.     {
45.         p = p -> next;
46.         q = q -> next;
47.     }
48.     if (q != NULL)
49.     {
50.         // 4.A: please fix the code
51.         p -> next = 4.A;
52.         free (q);
53.     }
54.     return head; /* note the return value */
55. }
56.
57. void List_destroy(Node * head)
58. {
59.     while (head != NULL)
60.     {
61.         Node * p = head -> next;
62.         free (head);
63.         head = p;
64.     }
65. }
66.
67. void List_print(Node * head)
68. {
69.     while (head != NULL)
70.     {
71.         printf("%d ", head -> value);
72.         head = head -> next;
73.     }
74.     printf("\n");
75. }
76.

```

```

77.  int main(int argc, char * argv[])
78.  {
79.      if (argc < 2) { return EXIT_FAILURE; }
80.
81.      Node * head = NULL;
82.      Node * ptr = NULL;
83.      int val;
84.      for (int i = 1; i < argc; i++)
85.      {
86.          val = (int) strtol(argv[i], NULL, 10);
87.          head = List_insert (head, val);
88.      }
89.
90.      List_print(head); // 4.B: What is the output?
91.
92.      head = List_insert(head, 77);
93.      ptr = List_search(head, 56);
94.      List_print(ptr); // 4.C: What is the output?
95.
96.      head = List_delete(head, 56);
97.      List_print(head); // 4.D: What is the output?
98.
99.      // 4.E: What is the output?
100.     if (List_search(head, 77) != NULL)
101.     {
102.         printf("found\n");
103.     }
104.     else
105.     {
106.         printf("not found\n");
107.     }
108.
109.     List_destroy(head);
110.     return EXIT_SUCCESS;
111. }

```

The executable program's name is q4prog.

The program is executed in a terminal using `./q4prog 56 12`

Please fix the code in 4.A, and write the output for 4.B - 4.E. Hint: please note the arguments for each function call in the main function.

Answer:

4.A: q -> next

4.B: 12 56

4.C: 56

4.D: 77 12

4.E: found